Experiment No. 9
ERC-20 Token Creation in Remix IDE

Aim

To create an ERC-20 token using Solidity in Remix IDE and deploy the token contract to a selected testnet using a MetaMask wallet, and to study the basic ERC-20 functions, events and attributes.

Theory

An ERC-20 token is a fungible blockchain token implemented through a common interface for token balances, transfers and allowances. The `MyToken` contract defines the public attributes `name`, `symbol`, `decimals` and `totalSupply`, together with the `balanceOf` and `allowance` mappings. It emits `Transfer` and `Approval` events when balances or spending permissions change.

- `transfer` – sends tokens from the caller to another address.
- `approve` – grants a spender an allowance.
- `transferFrom` – moves tokens using an approved allowance.
- **Constructor** – creates the initial supply and assigns it to the deployer.

Remix IDE compiles and deploys the contract; MetaMask approves the transactions on the selected testnet. ERC-20 represents fungible tokens, whereas ERC-721 is meant for non-fungible tokens (individually distinguishable assets).

Requirements

- Computer with a web browser and internet connection.
- Remix IDE (`https://remix.ethereum.org`) with the LearnEth plugin.
- MetaMask wallet connected to a testnet (Sepolia), funded using a faucet.
- Solidity compiler 0.8.34.

Procedure

Step 1: Study the ERC-20 course

Open the LearnEth Solidity tutorials in Remix IDE and go through the six chapters of the ERC-20 Token course.

Step 2: Set up MetaMask

Set up a MetaMask wallet, select a supported testnet and fund it with test ETH from a faucet.

Step 3: Create and compile

Create `MyToken.sol`, enter the contract and compile it with a compatible Solidity compiler (Figure 1).

Step 4: Deploy

Select the browser-wallet (Injected Provider) environment and connect MetaMask to the testnet. Enter the initial token supply and confirm the deployment transaction in MetaMask (Figures 2–3).

Step 5: Inspect the token

Inspect the deployed contract and its transaction details, and call the public functions to read the token name, symbol, decimals, total supply and balances (Figures 4–6).

Code

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

contract MyToken {

   string public name = "Jay Token";
   string public symbol = "JAY";
   uint8 public decimals = 18;

   uint256 public totalSupply;

   mapping(address => uint256) public balanceOf;

   mapping(address => mapping(address => uint256)) public allowance;

   event Transfer(
       address indexed from,
       address indexed to,
       uint256 value
   );

   event Approval(
       address indexed owner,
       address indexed spender,
       uint256 value
   );

   constructor(uint256 initialSupply) {
       totalSupply = initialSupply * 10 ** decimals;
       balanceOf[msg.sender] = totalSupply;

        emit Transfer(address(0), msg.sender, totalSupply);
   }

   function transfer(
       address to,
```

```
        uint256 amount
    ) public returns (bool) {

        require(
            balanceOf[msg.sender] >= amount,
            "Insufficient balance"
        );

        balanceOf[msg.sender] -= amount;
        balanceOf[to] += amount;

        emit Transfer(msg.sender, to, amount);

        return true;
    }

    function approve(
        address spender,
        uint256 amount
    ) public returns (bool) {

        allowance[msg.sender][spender] = amount;

        emit Approval(msg.sender, spender, amount);

        return true;
    }

    function transferFrom(
        address from,
        address to,
        uint256 amount
    ) public returns (bool) {

        require(
            balanceOf[from] >= amount,
            "Insufficient balance"
        );

        require(
            allowance[from][msg.sender] >= amount,
            "Allowance exceeded"
        );

        balanceOf[from] -= amount;
        balanceOf[to] += amount;

        allowance[from][msg.sender] -= amount;

        emit Transfer(from, to, amount);

        return true;
    }
}
```

Output

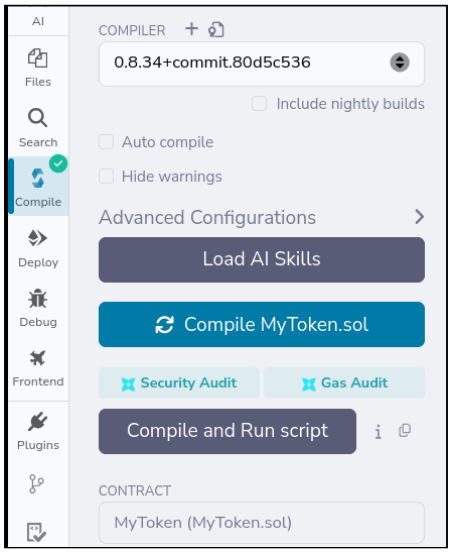


Figure 1: Compiler panel in Remix IDE – `MyToken.sol` (0.8.34)

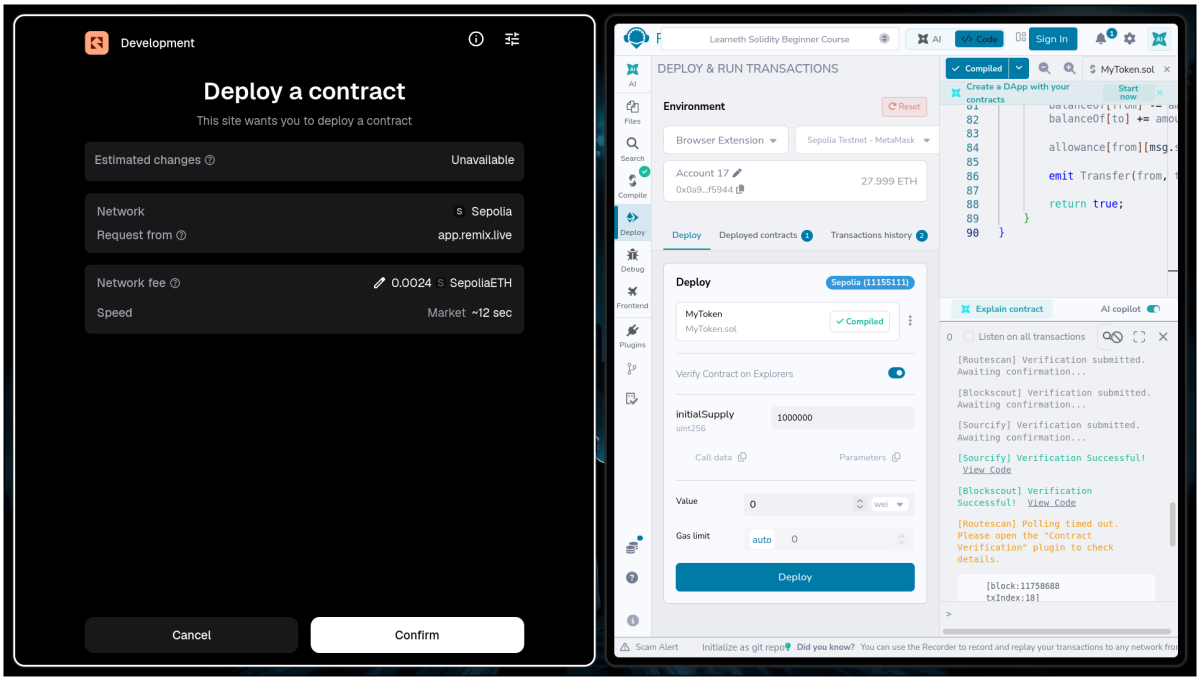


Figure 2: MetaMask “Deploy a contract” request and Remix deployment panel (initial supply)

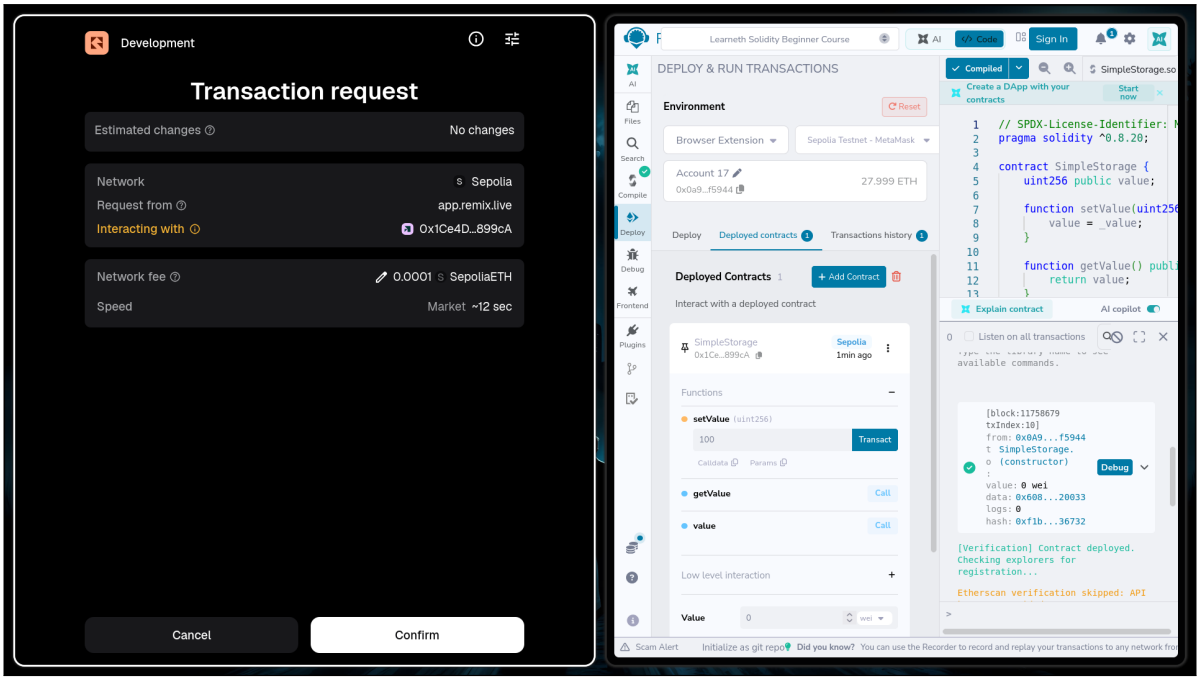


Figure 3: MetaMask transaction request on Sepolia

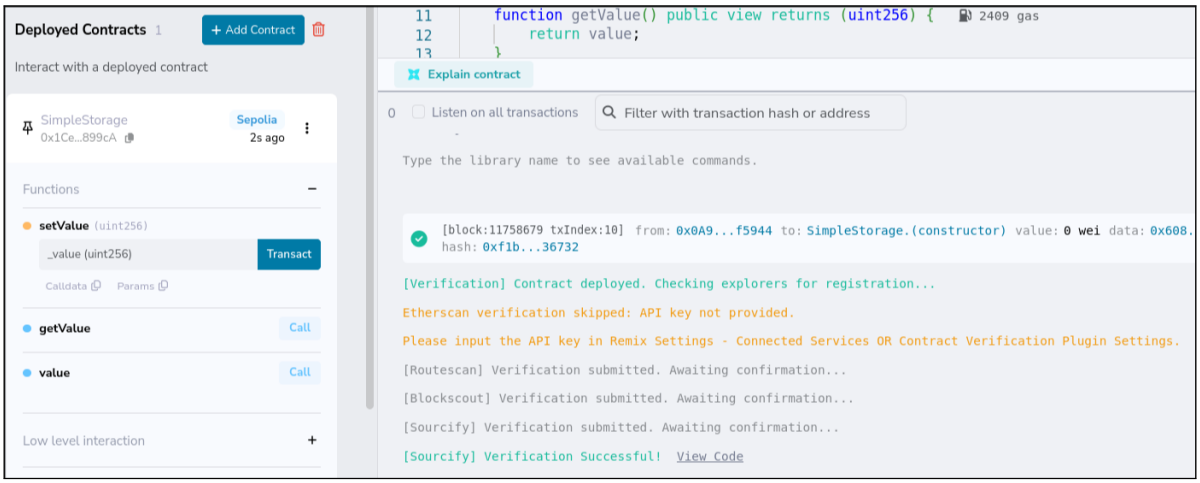


Figure 4: Deployed Contracts panel – contract verification successful

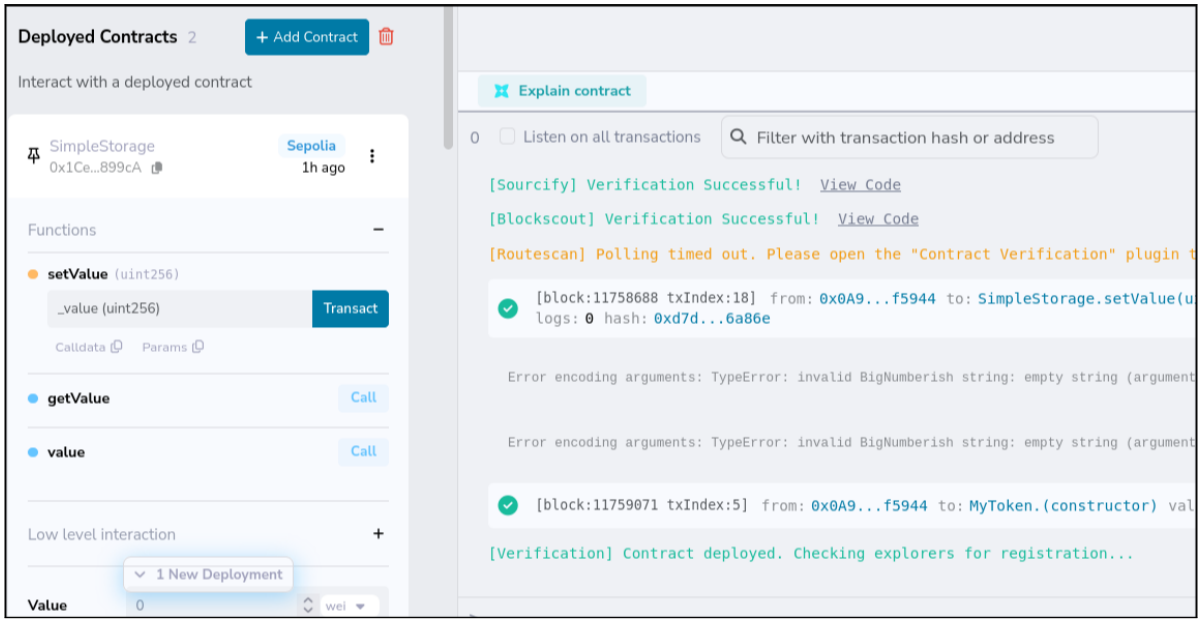


Figure 5: `MyToken.constructor` transaction confirmed in the Remix console



Figure 6: `MyToken` public attributes and call results (name, symbol, decimals, totalSupply)

Conclusion

The `MyToken` Solidity contract was created with ERC-20 token attributes, balance and allowance mappings, transfer and approval events, and the transfer functions. It was compiled in Remix IDE and deployed to the Sepolia testnet through MetaMask; the deployed contract and its public attributes were inspected. The experiment demonstrated how a fungible token contract is created and deployed using Remix IDE and a MetaMask-connected testnet.

Experiment completed successfully.